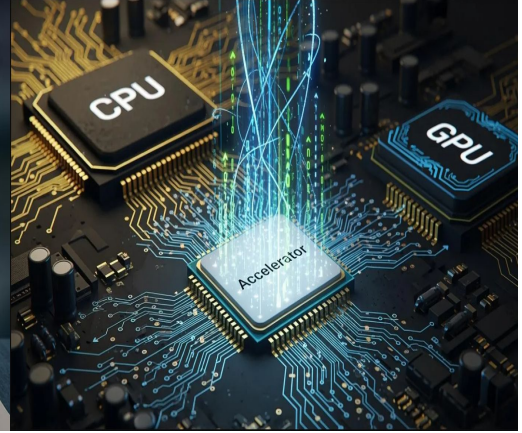
COMP4601: Team HOG Riders



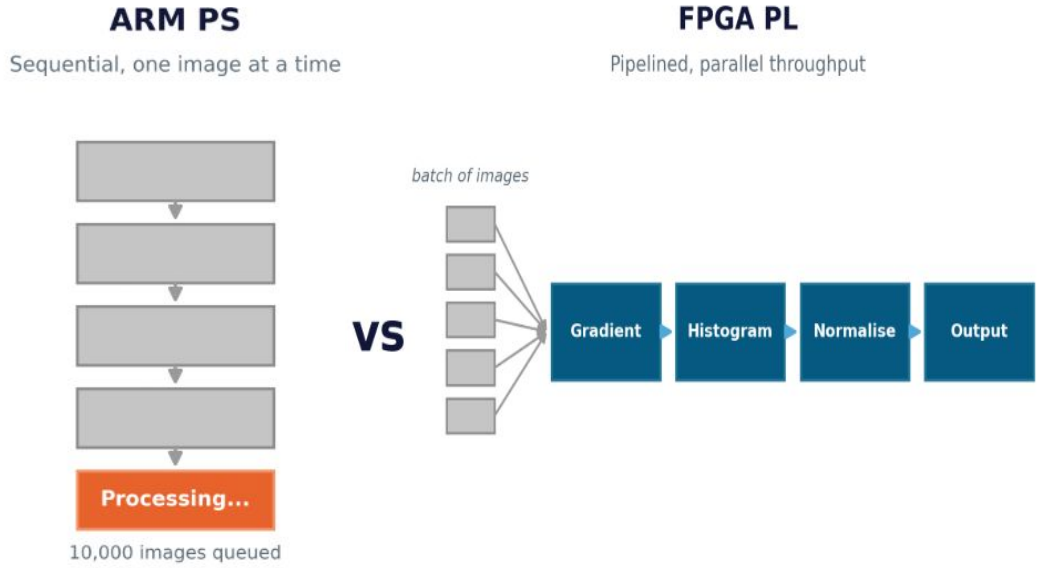
COMP4601 Project: Accelerated Histograms of Gradients

Name	Email	zID
Samyak Diwan	z5611048@ad.unsw.edu.au	z5611048
Abhiram Yanamandra	z549219@ad.unsw.edu.au	z549219
Srikaran Gomadam	z5478391@ad.unsw.edu.au	z5478391
Rhythm Nath	z5417295@ad.unsw.edu.au	z5417295

UNSW SYDNEY

Problem outline

- HoG is the standard feature extractor for classical (non-DL) image classification pipelines
- Computing it in software processes each pixel's gradient and histogram sequentially slow on an embedded ARM processor
- These operations are simple and repetitive: a natural fit for parallel hardware



Can FPGA-accelerated HoG match or exceed real-time throughput on a 10,000-image batch on the Kria KV260?

Project Aims

01

Software Baseline

C/C++ HoG implementation verified against reference output

02

HLS Acceleration

Vitis HLS on Kria KV260 with pipeline, unroll and partition pragmas

03

Benchmark

Latency and throughput vs ARM PS across 10,000 MNIST images

04

Resource Tradeoff

Speedup gained vs LUTs, BRAM and DSPs consumed

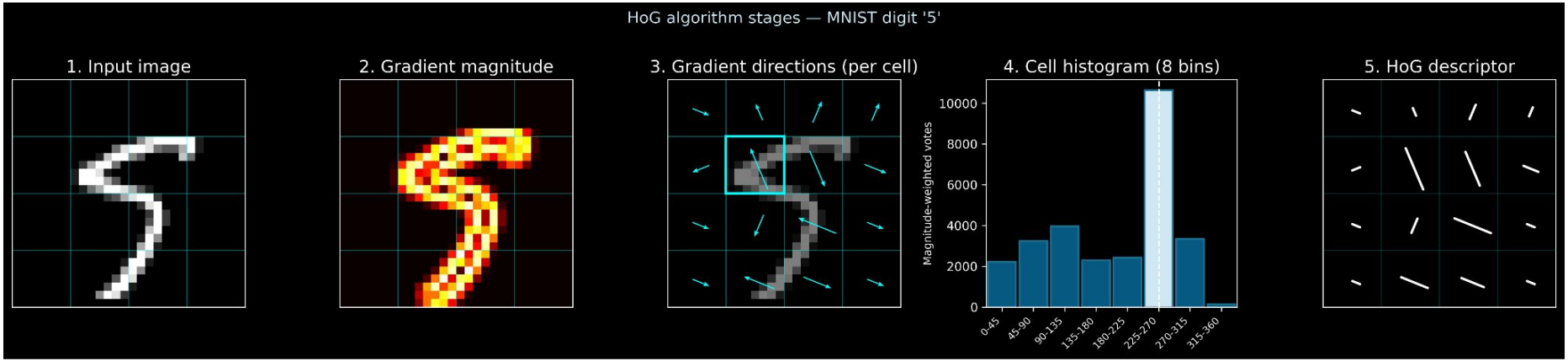
10,000
images benchmarked

3
HLS optimisation techniques

2
metrics: latency + throughput

Algorithm to be accelerated

HoG algorithm stages visualised on MNIST digit '5' (28×28, 8-bin, 4×4 cell grid)



Investigation so far

01

Studied the HoG Algorithm

Reviewed the full HoG pipeline: Sobel gradient computation per pixel, magnitude-weighted 8-bin histogram voting per cell, L2 block normalisation across 2x2 cell blocks, and final descriptor output. Confirmed the 4x4 cell grid structure for 28x28 MNIST images and identified each stage as a candidate for parallelism.

02

Identified Parallelism Opportunities

Gradient computation is fully data-parallel across pixels, making it a strong candidate for loop pipelining to II=1. Histogram accumulation has a read-after-write dependency that breaks naive pipelining; local per-cell accumulators are planned as the mitigation. Block normalisation involves sqrt/division, which may require fixed-point approximation to avoid exhausting DSPs.

03

HLS Optimisation Research

- Researched HLS acceleration techniques applicable to image processing: loop pipelining (targeting II=1), loop unrolling for histogram accumulation, and array partitioning to resolve memory port bottlenecks
- Identified read-after-write hazard in the histogram accumulation loop as a primary pipelining obstacle; local per-cell accumulators proposed as mitigation

04

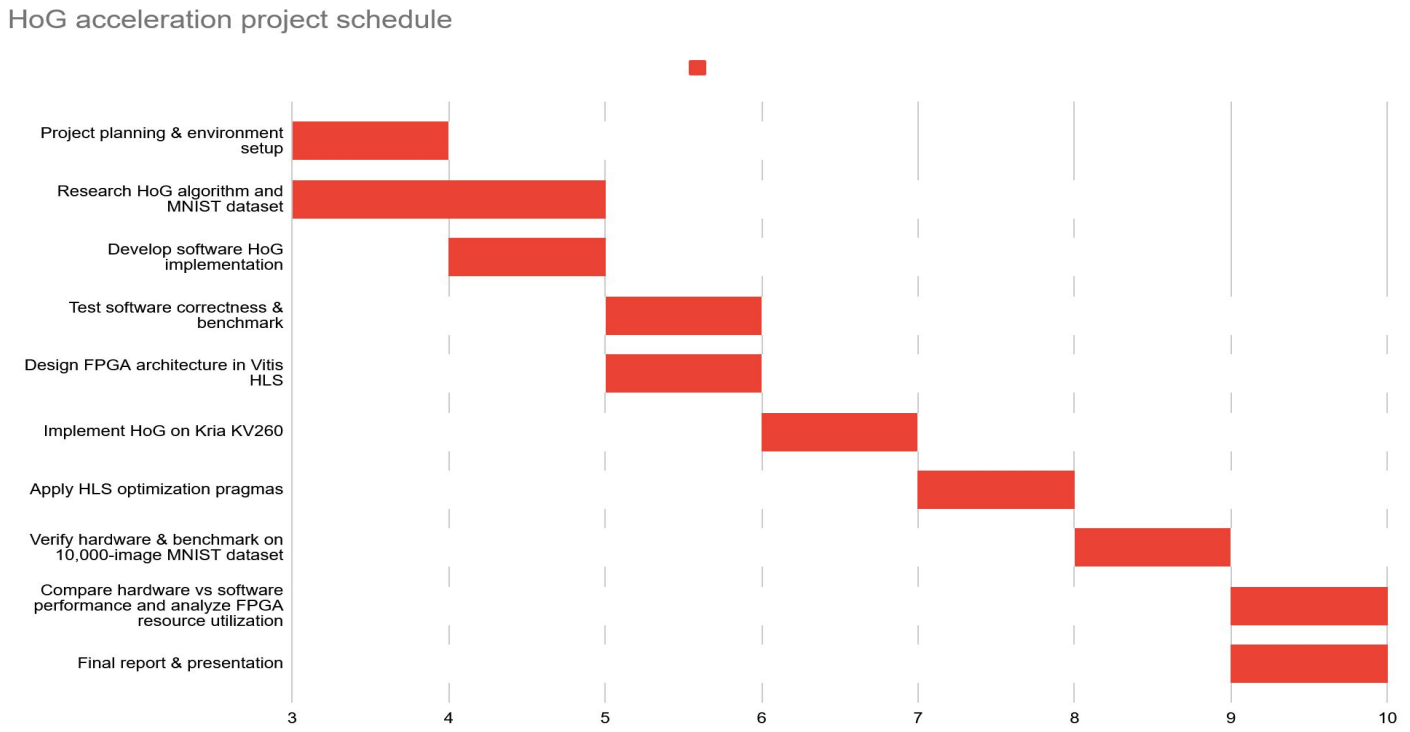
Studied KV260 & Reviewed Reference Material (Including Vitis Vision)

- Reviewed the Zynq UltraScale+ architecture and further explored Vitis HLS
- https://github.com/nikkatsa7/HOG_Zedboard
- https://ijres.iaescore.com/index.php/IJRES/article/view/21088
- https://xilinx.github.io/Vitis_Libraries/vision/2019.2/index.html

Risks & Contingencies

Risk	Mitigation
Histogram read-after-write hazard breaks HLS pipelining	Process one cell at a time using local per-cell accumulators before writing to output array
DMA transfer overhead dominates on 28×28 images	Batch multiple images per transfer never one image per DMA call
Block normalisation sqrt/division exhausts DSPs	Fall back to L1 normalisation or fixed-point sqrt approximation if resources are tight
AXI-DMA board integration takes longer than expected	Start board bring-up before Week 8 dont leave it until the end
Interface mismatch between the two HLS cores	Lock data type, bit width and AXI stream protocol in writing before Week 6

Project Plan



Roles

01

Software Baseline

Samyak Diwan

- C/C++ HoG implementation on ARM PS
- Profile to find compute hotspots
- Generate reference descriptors for verification

02

Verification & Testbench

Rhythm Nath

- C testbenches for all HLS cores
- Validate HLS C-sim vs software reference
- Accuracy validation on full MNIST batch

03

HLS Core Developer

Srikaran Gomadam

- Port HoG pipeline to Vitis HLS
- Apply pipeline, unroll, partition pragmas
- Resolve histogram read-after-write hazard

04

Host Code & Integration

Abhi Yanamandra

- ARM PS driver and AXI-DMA setup
- Image batching strategy
- Board bring-up and benchmarking

Dependency flow — each role unblocks the next